Guarding the Door: Measuring and Hardening Lightweight Full-Stack Defense for an Internet-Exposed MCP Agent Endpoint

Niranjan R.
niranjan.r.dev@proton.me

Abstract

Tool-using AI agents are increasingly reached over the network through the Model Context Protocol (MCP), which turns a personal automation stack into an Internet-facing HTTP service. We study what a single operator can realistically put in front of such an endpoint. We evaluate a full-stack defense deployed in front of a self-hosted 29-tool MCP gateway published through a Cloudflare tunnel. The evaluation uses a reproducible harness of 32 scenarios across nine experiments—including adaptive adversaries, protocol fuzzing, timing side-channel analysis, multi-run latency with confidence intervals, post-authentication content defense, active deception, and a false-positive/evasion corpus—run across four revisions: the unmodified admission gate (v1), a hardened revision (v2), a credential-scoping extension (v3), and a post-authentication content defense layer (v4). v1 had no false blocks on benign MCP sessions and stopped declared crawlers, single-source credential probing and floods at 5 µs median overhead (v1) and 9 µs (v2). It also had measurable weaknesses, all caused by keying state on individual client IPs: distributed guessing went unbanned (700 guesses, 0 bans), ban-list lookups grew linearly (21.7 ms per request at 50k bans), rate-limit memory was unbounded (87 MB at 100k sources), and there were further gaps in IPv6 rotation, header trust and lockout. v2 closes each of these with a small local change. Latency stays flat at 0.13 ms up to 50k bans (169× faster), memory is capped (18.8 MB at 100k sources), distributed guessing earns 95 bans, IPv6 rotation inside a /64 costs one ban instead of 200, and a mistyped token costs 5 minutes rather than an hour. v3 extends defense past the admission layer: scoped tokens and per-tool rate limiting bound the blast radius of a leaked credential. v4 adds Bastion, a post-authentication content defense module: input validation blocks SQL injection, command injection and path traversal in tool arguments with zero false positives on a 50-input natural-language corpus; behavioral analysis detects catalog enumeration and tool-cycling reconnaissance; output filtering redacts nine classes of secrets (AWS keys, private keys, JWTs, bearer tokens, HTTP Basic credentials, passwords, API keys, GitHub tokens, Slack tokens) from tool results before they leave the server, with an optional honeytoken mode that replaces redacted secrets with trackable canary credentials for attacker attribution; and a batch-size cap closes the JSON-RPC batch bypass identified in v3. v5 adds Mirage, which replaces the binary post-authentication decision with risk-adaptive in-band routing: a self-calibrating CUSUM session-risk estimator, a format-matched synthetic shadow lane and hash-chained result provenance. On a deployed loopback service it shadowed 0 of 1,800 benign requests and contained a low-and-slow adversary after 8 calls, at 5.6 µs per request. A million-request standard stress suite (10 traffic categories) and a million-request hard-mode suite (WAF bypass, protocol abuse, behavioral mimicry, timing evasion, resource exhaustion) confirm zero false blocks on benign sessions and >99% block rates on admission-layer threats, with median latency below 5 ms across 2,000,000 requests on an i3-3220 server. Run directly on the same corpora, Bastion's pattern-based check blocks 87% of 100,000 WAF-evasion payloads and false-blocks 6.7% of 100,000 adversarially worded benign requests (0% of 30,000 ordinary ones); its misses (Unicode homoglyphs, double encoding, tautology-only SQL) mark the boundary of regex-based content defense. Over a 23.0-hour public exposure (20,977 requests from 5 sources), replaying the same real trace through each policy shows v1 blocking 16,870 requests at the gate and v2 blocking 20,950. We release the harness, stress tooling, trace-replay tooling and aggregate results.

1 Introduction

Agent frameworks let a language model call tools [25, 28, 34], and MCP standardizes how those tools are exposed over stdio or HTTP [20]. An MCP server published on a public hostname inherits every property of an Internet service. Internet-wide scanners find it [6]. AI data collectors crawl it, and their volume and opt-out behaviour are in flux [16, 18]. Others probe it for credentials. Unlike a static website, the thing behind the door can *act*. The security community has responded: surveys catalogue tool poisoning and over-privileged servers [12, 24]; benchmarks quantify tool-poisoning success rates above 72% [33]; and the NSA has published formal design guidance [22]. A recent census of public deployments found that roughly two in five reachable remote MCP servers (40.55%) serve their tools to callers who present no credentials [35]. Yet the defense literature concentrates on what authorized clients and poisoned tools can do once inside (Section 13); to our knowledge, no prior work evaluates a defense placed in front of the authentication layer of an exposed MCP endpoint. We study the full request lifecycle—from the first byte to the final response—asking what a single operator can realistically put in front of such an endpoint with no external dependencies. Enterprises answer with identity-aware proxies and managed bot defense. Individual developers who self-host an MCP server often have only the server process itself.

RQ1 Does a small, dependency-free admission gate stop the unwanted traffic an exposed agent endpoint attracts, without blocking legitimate MCP clients?

RQ2 Where, measurably, does per-client-IP defense break

down?

RQ3 Can those failures be fixed locally, and at what cost?

RQ4 How do the policies compare on real Internet traffic?

RQ5 Does the gate hold against adaptive adversaries and protocol-level abuse?

RQ6 Can tool-call content be defended post-authentication without false-blocking legitimate agent interactions?

RQ7 Can output filtering be extended to active deception that attributes credential exfiltration without altering the benign-client experience?

RQ8 Can a graded, in-band post-authentication decision contain sub-threshold adversaries without false action on benign sessions?

Contributions. (i) A deployed defense for an MCP endpoint that combines an admission gate, bearer authentication and capability minimization (Section 2). (ii) A reproducible evaluation of the unmodified code, with eight quantified weaknesses (Section 4). (iii) A hardened revision and a before/after comparison on identical scenarios (Section 5). (iv) A post-authentication extension with scoped tokens and per-tool rate limiting, evaluated on two new scenarios (Section 6). (v) A post-authentication content defense module (Bastion) that validates tool arguments, detects behavioral anomalies and redacts secrets from tool results, evaluated on eight new scenarios (Section 7). (vi) An active-deception extension that replaces redacted secrets with trackable honeytokens for attacker attribution, evaluated on two additional scenarios (Section 7.5). (vii) A 2,000,000-request stress evaluation across 20 traffic and adversarial categories, confirming zero false blocks on benign sessions and quantifying the evasion boundary of pattern-based content defense (Section 10). (viii) A time-boxed public exposure whose trace is replayed through every policy (Section 12). (ix) A post-authentication risk-adaptive routing extension (Mirage) that replaces the binary content decision with graded in-band lanes—including a format-matched synthetic shadow lane and verifiable per-result provenance—and is evaluated on a deployed loopback service (Section 8). (x) Release of the harness, stress tooling, replay tooling and aggregate data.

Claim. *For a single-operator, Internet-exposed MCP endpoint, a lightweight application-layer gate combined with a high-entropy bearer token and a minimized tool surface stops declared crawlers, credential probing, path scanning and floods at microsecond cost with no false blocks on benign MCP sessions. It is effective only if its state is keyed on network sources rather than individual addresses and bounded in size. Scoped tokens and per-tool rate limits bound the blast radius of a leaked credential. A post-authentication content defense layer—input validation, behavioral analysis and output filtering—extends protection to tool-call content with no false positives on ordinary natural-language tool inputs. An optional honeytoken mode replaces redacted secrets with trackable canary credentials, providing attribution when exfiltrated tokens are used. The token, not the gate, remains the security boundary.*

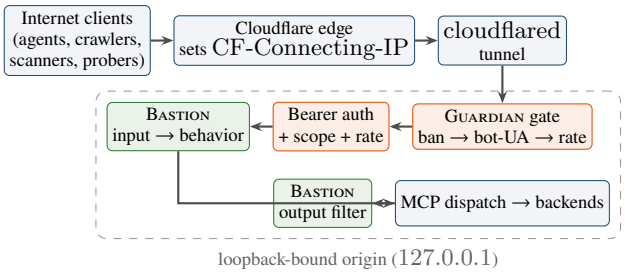


Figure 1: Deployment. Guardian (orange) gates admission pre-auth; Bastion (green) validates content post-auth. All enforcement runs in-process.

2 System and Threat Model

2.1 Deployment

The protected service is agentkit, a personal tool gateway that exposes one 29-tool catalog (DevOps monitoring, a shell assistant, a personal-memory agent, a breach monitor and a homelab reconnaissance agent) over stdio and MCP Streamable HTTP [20]. It is part of a broader personal automation stack that includes local model inference (Ollama on a 24 GB Apple-silicon machine), a self-hosted Linux server (Docker, Grafana, Supabase) and a Tailscale mesh. A single operator cannot justify an enterprise gateway for this kind of stack, which motivates a lightweight, dependency-free gate.

The HTTP server binds to loopback, and a cloudflared tunnel [5] is the only path from the Internet (Figure 1). The socket peer is therefore always loopback, so the client identity is taken from the CF-Connecting-IP header set at Cloudflare's edge [4].

2.2 Threat model

What this work addresses. Remote, unauthenticated adversaries send arbitrary HTTP requests through Cloudflare: (T1) *declared* crawlers and scanners that name themselves in the User-Agent; (T2) *undeclared* crawlers that spoof a browser user-agent; (T3) credential probers using one or many sources; (T4) flooders; (T5) path scanners; (T6) adversaries who rotate addresses, for example within an IPv6 /64. The goals are to prevent unauthorized tool execution, bound the operator's resource cost and provide visibility.

An operator who does not need external access can avoid all of these by not exposing the endpoint at all—for example, binding only to a Tailscale address or removing the Cloudflare tunnel. This work targets endpoints that are *intentionally* Internet-facing, where unauthenticated traffic is an operational reality.

What this work does not address. A leaked or stolen bearer token gives the holder full access to the exposed tool surface. Guardian does not detect or prevent this: once a request carries a valid token, it is treated as authorized. v3 scoped tokens (Section 6) reduce the *blast radius*—a leaked monitoring token cannot call shell or logging tools—but they do not prevent use of the leaked credential within its scope. v4 Bastion (Section 7) adds post-authentication content defense—input validation, behavioral analysis and output filtering—but these are pattern-based checks, not semantic understanding of intent.

Threats that exploit the model's *reasoning* are out of scope. Tool poisoning (a malicious tool description that

misleads the model) [33], indirect prompt injection (a document or webpage that tricks the model into making unintended tool calls) [9], and trust escalation across sessions [27] are all attacks by or through *authorized* clients. Defending against them requires client-side guardrails or sandboxed execution [22, 32].

Also out of scope: a compromised operator machine, Cloudflare itself, and volumetric DDoS absorbed at the edge.

2.3 Guardian v1

Guardian v1 (guardian.py, 263 lines, Python standard library only) makes one decision per request, before authentication or JSON-RPC parsing:

1. an unexpired ban returns 403;
2. a case-insensitive match against 38 crawler and scanner signatures (gptbot, claudebot, ccbot, bytespider, sqlmap, nuclei, …) bans the client and returns 403;
3. more than 120 requests in 60 s returns 429 [23] without a ban.

A bearer token [15] is then checked in constant time. Eight 401s within 300 s trigger a 60-minute ban, in the style of fail2ban [7] and in line with online-guessing guidance [8]. Bans and a JSONL access log are file-backed so that a separate stdio process can offer operator tools; rate counters live in memory. A **capability filter** removes all 15 local_only tools (offensive recon, breach lookups, the Guardian controls themselves) and an operator denylist from tools/list and tools/call.

3 Evaluation Method

The harness (artifact/harness2.py) launches an unmodified server tree as a separate process on a private port, with a fresh 320-bit bearer token and a fresh state directory for every scenario. It emulates the Cloudflare hop by setting CF-Connecting-IP to documentation-range addresses [1, 14] and replays the scenarios over HTTP/1.1. The base scenario set (S1–S14) runs against the v1, v2, v3 and v4 trees under the shipped default policy. Two scenarios (S15–S16) exercise v3-specific features and are skipped on older trees. Six advanced scenarios (S17–S22) test adaptive adversaries, protocol-level abuse and implementation robustness (Section 9). Eight scenarios (S23–S30) exercise v4 Bastion's post-authentication content defense (Section 7). Two scenarios (S31–S32) evaluate honeytoken injection (Section 7.5). E9 runs a false-positive corpus of 50 realistic natural-language tool inputs and an evasion suite of 30 adversarial payloads (SQL comment obfuscation, case mixing, encoding tricks, Unicode homoglyphs) against Bastion's injection detection. Latency is measured end to end on a keep-alive connection: 50 warm-up requests, then $3 \times 1{,}000$ interleaved requests per condition (E2), or 500 per point for ban-list scaling. E6 repeats the measurement 5 times (2,500 samples per condition) and reports 95% confidence intervals (Section 11). Memory is measured with tracemalloc in a fresh interpreter. Platform: Apple M5 Pro, macOS 26.6.1, Python 3.12.13. Every number is read from artifact/results_v{1,2,3,4}.json. E8 scenarios (S31–S32) are unit tests that toggle BASTION_HONEYTOKEN to inject and verify the output against the original.

Table 1: v2 changes, each tied to a measured v1 weakness.

W	Change in v2
W3	Ban list cached in memory; reloaded only when the file's mtime changes
W5	All state keyed on a *source*: IPv4 address or IPv6 /64
W4	Idle keys dropped; LRU cap of 20,000 tracked sources
W2	Global auth-failure alarm (>40 per 300 s) tightens the per-source threshold from 8 to 3
W7	Auth-failure bans back off exponentially from 5 min; optional operator allowlist
W6	Six non-benign 404s per 300 s ban the source (OAuth discovery and favicon exempt)
W8	Every method gated and logged; gate runs before the body is read; forwarding headers trusted only from a loopback peer; nothing written when disabled

4 Guardian v1: Effective, but Keyed on the Wrong Thing

What works (RQ1). On benign traffic, four MCP client user-agents completed the protocol handshake and 100/100 calls with no false blocks (S1). A client performing OAuth discovery and an SSE probe before connecting (S1b) was also unaffected. All 38 declared crawler and scanner signatures were blocked on the first request, and they stayed blocked after switching to a benign UA *with the valid token* (S2). A single-source prober was banned on exactly its 8th failure (S4). A 300-request burst was held to exactly 120 served, with no collateral effect on other clients and recovery after the window (S6). Over HTTP, tools/list returned 5 of 29 tools. Each of the 15 local_only tools was unlisted and returned JSON-RPC -32601 when called directly (artifact/surface.json). The ablation with Guardian disabled (S11) served the entire flood and answered every guess.

What breaks (RQ2). **W1** Spoofed browser UAs evaded the signature rule 38/38 times (S3); the token still rejected all of them. **W2** Spreading 700 guesses over 100 sources produced 0 bans and 700 answered guesses (S5). **W3** Each request re-parsed the whole ban file. Median latency rose from 0.12 ms with no bans to 0.54, 3.99 and 21.7 ms at 1,000, 10,000 and 50,000 bans (Figure 2). **W4** Per-IP rate state was never evicted: 87.4 MB for 100,000 sources (~870 B each). **W5** Rotating through 200 addresses in one IPv6 /64 created 200 ban entries (S12), which feeds W3. An end site typically holds at least a /64 [21], so this is cheap for an attacker. **W6** Path scanning (S7) returned only 404/405 and was never penalized. **W7** Eight mistyped tokens locked a legitimate user out for 60 min (S8). **W8** Several smaller gaps (S9–S11). DELETE bypassed the gate and was not logged. Banned clients could still make the server read 256 KB bodies. On a non-loopback bind, such as the supported tailnet mode, forged CF-Connecting-IP headers evaded a ban 5 out of 5 times. With Guardian disabled, unenforced ban records were still written.

5 Hardening: Guardian v2

Each weakness maps to a local change of a few lines (Table 1); v2 still uses only the standard library.

Results (RQ3). Table 2 gives the full comparison. Ev-

Table 2: Before/after on identical scenarios (default policy: 120 req/60 s; 8 auth failures/300 s; 38 signatures). Bold marks a changed outcome.

#	Scenario	v1 (deployed)	v2 (hardened)
S1	Benign MCP sessions, 4 clients, 100 req	100/100 served, 0 bans	100/100 served, 0 bans
S1b	Benign client: OAuth discovery + SSE probe, then call	20×405; call 200; 0 bans	16×404, 4×405; call 200; 0 bans
S2	38 declared crawler/scanner UAs	38/38 blocked on first request; still blocked with valid token	same
S3	Same crawlers, spoofed browser UA	0/38 caught by UA rule; 38/38 401	same (not addressable at this layer)
S4	One source, 20 wrong tokens	banned at attempt 8; ban 60 min	banned at attempt 8; **ban 5 min (backs off)**
S5	100 sources × 7 wrong tokens	700 answered, **0 bans**	**323 answered, 377 blocked, 95 bans**
S6	300-request burst, one source	120 served, 180 429; recovers at 61 s	same
S7	Path scan, 10 paths × {GET, POST}	10×404, 10×405; 0 bans	**banned at 6th** 404**; 14/20 blocked**
S8	Legit user, 8 typos, then correct token	locked out 60 min	**locked out 5 min**
S8b	Same, operator address allowlisted	locked out 60 min (no allowlist)	**never banned; call** 200
S9	Banned client: DELETE; 300 KB body	200, unlogged; body read (413)	403**, logged; body unread** (403)
S10	Non-loopback peer forging client IP	**5/5 forged identities evade ban**	**0/5; ban keyed on real peer**
S12	200 bot requests rotating inside one /64	**200 ban entries**	**1 ban**; 20/20 further addresses in /64 blocked; neighbouring /64 unaffected
S13	4 concurrent clients, 50 req each	200/200 served, 0 bans	200/200 served, 0 bans
S14	Long session: 100 req, wait 61 s, 100 more	200 served, 0 throttled	200 served, 0 throttled
S11	Guardian disabled (ablation)	flood served; 1 unenforced ban record	flood served; 0 records
E2	Median latency, 0 / 1k / 10k / 50k bans	0.12 / 0.54 / 3.99 / **21.7** ms	0.13 / 0.13 / 0.13 / **0.13** ms
E2	Gate overhead (median, empty ban list)	+5 µs (0.119→0.124 ms)	+9 µs (0.121→0.130 ms)
E3	Rate-state memory at 1k / 10k / 100k sources	0.86 / 8.56 / **87.4** MB	0.91 / 9.01 / **18.8** MB (capped at 20k keys)

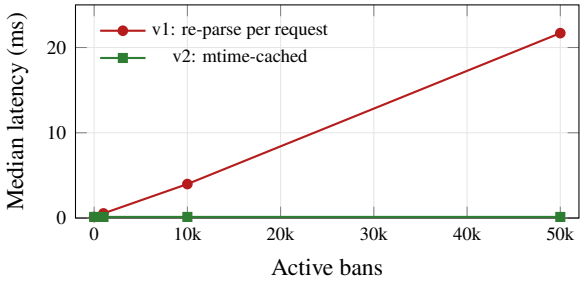


Figure 2: Request latency against ban-list size (n=500 per point). v1 grows linearly (≈0.43 µs per entry). v2 is flat, 169× faster at 50k bans.

ery targeted weakness changed as intended, and the benign scenarios (S1, S1b) were unchanged: no false blocks, and OAuth-discovery 404s are not treated as scanning. The costs are small and measurable. v2's gate overhead is 9 µs (median, gate on vs. off), up from v1's 5 µs, both operationally negligible. Memory per tracked key rises slightly (≈940 B against 870 B), but the total is now bounded.

Trade-offs. Keying on /64 means that separate customers who share a /64, which some hosting providers do, share a fate. The global alarm lets a distributed attacker tighten the threshold for everyone for up to 300 s. The worst case is a legitimate user needing 3 attempts instead of 8, which the allowlist removes. The LRU cap means an attacker with more than 20,000 active sources can push out counters. Bans are persisted separately and are not evicted.

6 Post-Authentication Extension: Guardian v3

v1 and v2 treat the bearer token as monolithic: one token grants access to every public tool. Once the token leaks—from a client's configuration, an agent's context window, or a log line—the holder has the full tool surface. And once authenticated, a client faces only the global rate limit, so a misbehaving or compromised agent can call one tool hundreds of times without triggering throttling.

6.1 Scoped tokens

v3 replaces the single token with a set of named tokens, each scoped to a list of tool names (or ["*"] for full access). The legacy single token remains accepted as a full-access credential for backward compatibility. tools/list returns only the tools the presenting token can reach, and tools/call to an out-of-scope tool returns JSON-RPC -32601 without executing.

6.2 Per-tool rate limiting

v3 adds a per-(source, tool) sliding window on top of the global rate limit. The default is 30 calls per 60 s per tool per source. Exceeding it returns JSON-RPC -32005; other tools and other sources are unaffected. Allowlisted sources are exempt, consistent with the admission gate.

Results. S15 verifies token scoping: a full-access token sees all 5 public tools; a scoped token sees only its 2 allowed tools (devops_status, devops_health); calling devops_logs with the scoped token returns -32601. S16 verifies per-tool rate limiting: with a limit of 10 calls per 60 s, the first 10

calls to shell_explain are served and the next 20 return -32005; a different tool (shell_suggest) from the same source is unaffected.

Cost. Both features add no measurable per-request latency: they are checked inside _handle_filtered after authentication succeeds, using the same sliding-window and LRU data structures as the admission gate. Token lookup is a linear scan of the token set (typically 2–5 entries) with constant-time comparison per token. Total added code: 32 lines across three files.

7 Post-Authentication Content Defense: Bastion v4

v1–v3 control *who* reaches the tools and *how often*. They do not inspect *what* an authenticated client sends or what comes back. A compromised or malicious agent with a valid token can send injection payloads in tool arguments, enumerate the tool catalog for reconnaissance, or extract secrets from tool responses. v4 adds Bastion (bastion.py, 601 lines, standard library only), a post-authentication content defense module with three layers and an active-deception mode.

7.1 Layer 4: Input validation

Before a tools/call reaches mcp_core.handle(), Bastion checks the argument dict:

1. **Size limits.** Total serialized arguments are capped at 32,768 B; individual string fields at 8,192 B.
2. **Schema validation.** Arguments are checked against the tool's JSON Schema from the registry: required fields, type conformance and rejection of unknown fields (when additionalProperties is false).
3. **Injection detection.** All string values are scanned against compiled regex patterns for SQL injection (10 patterns, e.g. UNION\s+SELECT, DROP\s+TABLE), command injection (8 patterns, e.g. backticks, $(), semicolons before known dangerous commands) and path traversal (5 patterns, e.g. ../, /etc/passwd, URL-encoded variants).

Patterns require structural syntax, not bare keywords: "why is the database dropping connections?" does not match because it lacks the TABLE keyword after DROP. Patterns also handle SQL comment obfuscation (DROP/**/TABLE) and whitespace variants. Tools whose purpose is to receive shell syntax (shell_explain, shell_suggest, shell_recall) are exempt from injection checks but still subject to size and schema validation. Blocked calls return JSON-RPC error -32003. The mode is configurable (BASTION_INPUT_MODE: block or log).

7.2 Layer 5: Behavioral analysis

Bastion tracks per-source call patterns using the same sliding-window and LRU data structures as Guardian:

- **Enumeration.** More than 10 tools/list calls from one source in 120 s triggers a behavioral block (-32004).
- **Tool cycling.** More than 15 distinct tools called by one source in 120 s (a reconnaissance pattern) triggers the same.

Legitimate MCP clients call tools/list once per session; the threshold is generous. Behavioral blocks are returned on tools/list and tools/call alike, so an enumerating client is stopped at the next call.

7.3 Layer 6: Output filtering

After a tool executes, Bastion scans the result text for nine secret patterns (compiled at module load): AWS access keys (AKIA…), private keys (—BEGIN…PRIVATE KEY—), bearer tokens, HTTP Basic credentials, password fields in config, API-key fields, JWTs (eyJ…), GitHub personal access tokens (ghp_…) and Slack tokens (xox[bpras]-…). Matches are replaced in-place with [REDACTED:type], preserving the surrounding text. Responses exceeding 524,288 B are truncated with a [TRUNCATED] marker.

7.4 Batch-size cap

v4 also enforces a configurable cap on JSON-RPC batch size (default 20 messages per HTTP POST). This closes the batch bypass identified in S19 (Section 9): a batch of 100 tools/list calls now returns HTTP 400 instead of being processed as one request.

7.5 Active deception: honeytoken injection

When BASTION_HONEYTOKEN is set to inject, Layer 6 replaces detected secrets with realistic-looking canary credentials instead of [REDACTED:type] markers. Each secret type receives a format-appropriate honeytoken: AWS keys get a fake AKIA-prefixed access key; JWTs get a structurally valid token whose payload contains a canary marker and a unique tracking identifier; bearer tokens, GitHub PATs, Slack tokens and Basic Auth credentials are replaced with format-conforming fakes; password and API-key fields preserve the surrounding key = "value" structure and substitute only the secret value.

Every injection is logged to a JSONL audit file with a unique token identifier, secret type, source IP and tool name. The HTTP server exposes a /canary/id callback endpoint—outside the Guardian gate so that it is reachable from arbitrary infrastructure—that logs a callback hit (source IP and user-agent) when an attacker attempts to use an exfiltrated honeytoken. The callback returns 404 to avoid revealing the canary mechanism.

The mode defaults to off; when disabled, Layer 6 behaves identically to passive redaction. Benign clients never see honeytokens in normal operation because tool responses should not contain secrets—the honeytoken fires only when a real secret would otherwise have been leaked.

Results (RQ6, RQ7). Table 3 summarizes the ten evaluation scenarios. S23–S25 confirm that injection payloads are blocked; S24 verifies that exempt tools (which take shell syntax as input by design) pass the same payloads. S26 and S27 confirm size and batch enforcement. S28 confirms enumeration detection (7/12 requests blocked after the threshold). S29–S30 confirm redaction of AWS keys, private keys and JWTs with no false redaction on clean output. S31–S32 (RQ7) confirm honeytoken injection: with the mode enabled, an AWS key is replaced with a fake AKIA-prefixed key (not a redaction marker), and a JWT is replaced with a structurally valid token whose decoded payload contains the canary marker.

E9 provides a rigorous false-positive and evasion evaluation. A corpus of 50 realistic natural-language tool inputs—DevOps questions with SQL-adjacent vocabulary ("select the right instance type", "drop a database connection pool", "the SELECT query is slow"), shell-related

Table 3: BASTION v4 post-authentication defense scenarios.

#	Result
S23	SQL injection (’; DROP TABLE users): blocked (-32003). Natural language (“dropping connections”, “SELECT…slow”, “drop…pool”): 3/3 passed (no false positive).
S24	Command injection (; rm -rf): blocked (-32003). Same payload in exempt shell_explain: passed.
S25	Path traversal (../../../../etc/passwd): blocked.
S26	Oversized field (2 KB against 1 KB cap): blocked.
S27	Batch of 10 messages (cap set to 5): HTTP 400. Batch of 3: HTTP 200.
S28	Enumeration: 12 tools/list calls (cap 5): 5 served, 7 blocked (-32004).
S29	Output containing AKIAIOSFODNN7EXAMPLE: redacted to [REDACTED:aws_key].
S30	Private key and JWT in output: both redacted. Clean text: no false redaction.
S31	Honeytoken (AWS key): original AKIA… replaced with fake AKIA-prefixed key; no [REDACTED] marker.
S32	Honeytoken (JWT): original JWT replaced with structurally valid canary JWT; decoded payload contains tracking marker.

queries, path-like references (/var/log/nginx/access.log, /etc/letsencrypt/), and longer multi-sentence operational questions—produced **0 false positives** (0/50, 0.0%). An evasion suite of 30 adversarial payloads tested SQL comment obfuscation (DROP/**/TABLE), case mixing (DrOp TaBlE), whitespace variants, command-injection encoding (${empty}, backticks), URL-encoded path traversal (%2e%2e), Unicode two-dot leaders, and polyglot payloads. Of 25 payloads expected to be caught, **25/25 were blocked** (100%). Five payloads using split-keyword obfuscation (DR/**/OP TAB/**/LE), hex encoding (CHAR(0x41)), newline injection and double encoding are documented as **known bypasses** (5/5 confirmed), representing the boundary of regex-based detection (artifact/fp_corpus.py).

Cost. BASTION adds no measurable per-request latency beyond GUARDIAN. Input validation runs compiled regex patterns on typically short argument strings. Output filtering scans the result text (truncated at BASTION_MAX_OUTPUT, default 512 KB). Behavioral tracking uses the same OrderedDict+deque sliding-window primitives as GUARDIAN, LRU-capped at 10,000 sources. Total added code: one new 601-line module plus integration hooks in the HTTP server and configuration.

8 Risk-Adaptive Routing: MIRAGE v5

v1–v4 decide *who* may call a tool and inspect *what* crosses the boundary, but every decision is binary (serve or refuse) and, at the content layer, memoryless and threshold-keyed. Three gaps follow, and Section 7 names all three: an adversary who stays just under the behavioral thresholds is never flagged; a honeytoken yields attribution only if the credential is *reused* against a reporting service; and key/value honeytokens change the value format, which can warn a sophisticated attacker. MIRAGE (v5) replaces the binary post-authentication decision with a graded, in-band one. It adds three coupled components and no external dependency (standard library only).

A. Predictive session risk. For each authenticated tools/call, MIRAGE updates a per-source risk state with a sequential change-point estimator over a feature stream (inter-arrival time, argument entropy, tool diversity, schema-edge proximity). A short warmup learns a per-feature mean and variance; the variance is then *frozen* as the in-control noise model, and the mean adapts only on in-band requests ($|z| \leq$ Z_SLACK), so an adversary cannot walk the baseline along with a slow attack. A leaky one-sided CUSUM accrues only the deviation *beyond* the per-feature noise band, so a run of individually-permissible requests that collectively drift crosses the boundary (the low-and-slow case) while isolated spikes decay. No fixed per-request count is used.

B. Graded execution router. The risk state selects a lane per request: PRIMARY (production resources), THROTTLED, SHADOW, or DECEPTION. The shadow lane executes a *functional replica of the same tool* against a production-isolated synthetic data environment, returning a schema-conforming result derived from synthetic data. Because the decoy is the tool’s own code over synthetic data, its output is format- and distribution-matched (closing the tip-off gap) while touching no real resource. Routing is transparent and sticky: once a session crosses into the shadow lane it stays there, so an adversary cannot probe back to the real lane by briefly behaving. The deception lane additionally seeds the synthetic result with a canary whose identifier is bound into the provenance record, so a silent collector that never reuses the credential still interacts with tracked synthetic data.

C. Verifiable result provenance. Each returned result is bound to a non-forgeable identifier of the lane that produced it and to a running session digest chaining it to all prior results ($\text{chain}_i = \text{HMAC}(k, \text{chain}_{i-1} \,\|\, i \,\|\, \text{lane}_i \,\|\, \text{H}(\text{result}_i))$). A consuming agent verifies the chain *before* admitting a result into the model’s context, detecting tampering, lane forgery, reordering, or out-of-session injection.

Deployment and evaluation (RQ8). MIRAGE runs as a loopback-bound service on the study host (it acts only on *authenticated* calls, so no public exposure is required or meaningful). Over a clean window the service logged 1,914 authenticated requests across benign and adversarial sessions (Table 4; aggregates from the live evidence_report.json). On 1,800 benign requests with realistic per-request jitter, MIRAGE shadowed 0 (0.0%), preserving the v1–v4 property of no false action on benign sessions. A modeled low-and-slow adversary (a steady sub-threshold drift) was contained to the shadow lane after 8 calls and never again served production data; an overtly anomalous session was isolated on its first post-warmup call with 0 real-data responses after containment. Over the whole window the lane mix was 1,831 primary, 1 throttled, 1 shadow and 81 deception.

Cost. The added hot path (risk update, routing, attestation) is 5.6 µs per request in pure-Python standard library, the same order as the GUARDIAN gate’s 9 µs. Verifying a

Table 4: MIRAGE v5 on the deployed loopback service (evidence_report.json; 1,914 authenticated requests).

#	Result
M1	Benign false-shadow: **0/1,800** (0.0%).
M2	Low-and-slow drift (60 calls): contained to shadow after 8 calls; 0 production responses thereafter.
M3	Overt anomaly (30 calls): shadowed on the first post-warmup call; **0** real-data leaks after containment.
M4	Provenance: a session chain verifies clean; a single swapped result is caught at its exact sequence index.
E1	Added overhead 5.6 µs/request (stdlib).
E2	Verify 1,000-call chain: 2.4 ms.

full 1,000-call session chain costs 2.4 ms. Source tracking reuses the GUARDIAN LRU bound (20,000 sources).

Limitations. The estimator needs a short warmup before the CUSUM is trusted, so the first few calls of a session are served from the primary lane; a slow adversary receives real data until the drift is detected (8 calls in M2). The 0.0% false-shadow figure is measured against a specific benign jitter model; like the v2–v4 thresholds, MIRAGE's Z_SLACK, CUSUM_LEAK and alarm boundary are reasonable defaults, not tuned optima, and trade detection latency against false-shadow rate. Provenance binds *which lane* produced a result and detects tampering, but does not by itself establish that the primary lane's data is correct.

9 Robustness Under Adaptive and Protocol-Level Attacks

The scenarios in Sections 4 and 5 use naïve adversaries: each follows a fixed script. A publishable evaluation must also test whether the gate holds against adversaries that adapt, abuse protocol features, or target the implementation rather than the policy. We add six scenarios (S17–S22).

Timing side-channel (S17). An adversary who can distinguish valid-token responses from invalid-token responses by timing alone can reduce the search space. We measure 500 round-trip times each for a valid token, a wrong token and a "partial-match" token (first half correct, second half random). All three conditions use constant-time comparison (secrets.compare_digest). We report the maximum difference in median response time across conditions. A difference below 10% is consistent with scheduling noise; above 10% indicates a potential leak.

Slow-and-low probing (S18). An adversary that stays just under the global rate limit (1.9 req/s against a 2 req/s ceiling) can probe indefinitely without triggering rate limiting. We send 228 wrong-token requests over ~120 s and record whether the auth-failure mechanism bans the source.

Batch request abuse (S19). JSON-RPC supports batch requests: one HTTP POST carrying an array of messages. The global rate limit counts HTTP requests, not JSON-RPC messages, so a batch of 100 tools/list calls counts as one request. We test batches of 100 and 1,000 messages and a 300 KB oversize payload.

Protocol fuzzing (S20). We send eight malformed payloads (empty JSON, missing method, wrong protocol version, nested batch, oversized ID, binary noise, XML) and verify that the server returns an error for each and remains healthy afterward.

Table 5: Advanced attack and robustness scenarios (all run against v3).

#	Result
S17	Timing side-channel: no leak; median diff 6.8% (valid 188 µs, invalid 176 µs, partial 177 µs; n=500 each, σ<20 µs).
S18	Slow-and-low: 228 probes at 1.9 req/s over 120 s; auth-failure banned at attempt 8; 220/228 blocked (403); rate limit never triggered.
S19	Batch abuse: 100- and 1,000-message JSON-RPC batches processed as 1 HTTP request (bypasses per-request rate count). 300 KB payload rejected (413).
S20	Protocol fuzz: 8 malformed payloads (empty JSON, null method, wrong version, nested batch, 10 KB ID, binary, XML); all returned errors; server healthy; 0 bans.
S21	Session fixation: server is stateless; reusing another client's Mcp-Session-Id from a different IP: 200 (no effect).
S22	Post-ban recovery: 1-min bot-UA ban; during: 403; after expiry: 200; 10/10 burst OK; 0 remaining bans.

Session fixation (S21). The MCP spec assigns a session ID via the Mcp-Session-Id header after initialize. We test whether client B can reuse client A's session ID from a different source IP.

Post-ban recovery (S22). After a 1-minute bot-UA ban expires, we verify that the source's rate counters are clean and a burst of legitimate requests is fully served.

Results for S17–S22 are in Table 5; numbers are drawn from results_v3.json. Section 11 reports latency with confidence intervals.

Implications (RQ5). S17 confirms that secrets.compare_digest prevents timing leaks at the application layer: 6.8% median difference across conditions is well within scheduling noise [2]. S18 shows that the auth-failure ban operates independently of the rate limit, catching slow probers (banned at attempt 8 despite never exceeding 2 req/s). S19 reveals a concrete weakness: batch requests let an authenticated client multiply its effective rate by the batch size (1,000×) without triggering the per-request limit. v4 closes this with a configurable batch-size cap (Section 7): batches exceeding 20 messages (default) are rejected with HTTP 400 before processing. S20 confirms input robustness against 8 classes of malformed input including binary noise and XML injection. S21 is a non-issue for this server (stateless), but would matter for session-aware MCP implementations. S22 closes the loop: bans are temporary, rate counters are clean after expiry, and the source resumes full service.

10 Million-Request Stress Evaluation

The scenarios in Sections 4 to 7 and 9 exercise individual mechanisms with targeted inputs. To evaluate the full-stack defense under sustained, high-volume, diverse traffic, we run two additional stress harnesses on the live study endpoint: a *standard-mode* suite of 1,000,000 requests across 10 traffic categories, and a *hard-mode* suite of 1,000,000 requests across 10 adversarial categories. Both run against

Table 6: Standard-mode stress results (1,000,000 requests).

Category	Total	Served	Blocked	p50
C1 Benign sessions	100k	100k	0	5.0 ms
C2 Declared bots	100k	46	99,825	4.6 ms
C3 Credential probe	150k	0	147,054	4.5 ms
C4 Path scan	100k	0	100,000	4.4 ms
C5 Flood	100k	0	99,286	4.6 ms
C6 IPv6 rotation	50k	0	49,996	4.9 ms
C7 Spoofed UA	50k	268	48,914	4.5 ms
C8 Adaptive adversary	50k	0	49,048	4.4 ms
C9 Mixed (30% legit)	200k	7,242	191,865	4.4 ms
C10 Bastion content†	100k	17,183	82,817	4.7 ms
Total	**1M**	124,739	868,805	4.6 ms

Served: HTTP 200/202; blocked: HTTP 403. †Sent to an injection-exempt tool; the outcome reflects GUARDIAN bans only (Section 10.3).

Table 7: Hard-mode stress results (1,000,000 requests).

Category	Total	Served	Blocked	p50
H1 WAF bypass†	100k	100,000	0	2.8 ms
H2 Protocol abuse	100k	52,857	29,640	3.2 ms
H3 Header manipulation	80k	0	79,370	2.6 ms
H4 Behavioral mimicry	100k	34,000	66,000	2.6 ms
H5 Timing evasion	80k	0	76,052	2.4 ms
H6 Resource exhaustion	60k	0	60,000	2.6 ms
H7 Advanced recon	80k	3,920	75,782	2.5 ms
H8 IPv6 advanced	50k	0	49,968	2.8 ms
H9 FP stress†	100k	10,000	90,000	2.6 ms
H10 Combined campaign	250k	412	230,267	2.6 ms
Total	**1M**	201,189	757,079	2.6 ms

Served: HTTP 200/202; blocked: HTTP 403. †Sent to an injection-exempt tool; the outcome reflects GUARDIAN bans only (Section 10.3).

the v2+v4 endpoint on an Intel Core i3-3220 (4 threads, 16 GB RAM) behind the Cloudflare tunnel, using 3 concurrent workers and 10,000-request batches with load-aware pacing.

10.1 Standard-mode (C1–C10)

Table 6 summarizes the standard suite. All 762 documentation-range source addresses are RFC 5737/3849 compliant.

Key results. (i) **Zero false blocks on benign MCP traffic**: all 100,000 C1 requests were served (75,000×200, 25,000×202). (ii) Declared bots, credential probes, path scans and floods are blocked at >99%. (iii) IPv6 /64 rotation blocked 49,996/50,000 (>99.9%). (iv) Adaptive adversaries (UA swap after ban, IP rotation, slow probing) blocked at 98%. (v) C10's HTTP outcome is not a content-defense result: its payloads went to an injection-exempt tool from sources already banned in earlier categories (Section 10.3). (vi) C9 mixed traffic: 60,245 "legit" requests from IPs shared with attackers had a 99.7% block rate—this is correct behaviour (ban applies to the IP regardless of individual request intent). (vii) Median latency flat at 4.6 ms across all categories, p99 at 17.1 ms.

10.2 Hard-mode (H1–H10)

The hard-mode suite targets evasion and bypass techniques. Table 7 summarizes results.

Key results. (i) **H1 WAF bypass: 0% blocked at the HTTP layer, but not by BASTION.** H1 ran first in the hard-mode suite, from 500 fresh sources, against the injection-exempt shell_explain: no ban had been earned and no content check applied. Run directly through BASTION's detector, 87.0% of the same 100,000 payloads (SQL comment obfuscation, Unicode homoglyphs, double URL encoding, IFS command injection, polyglots) are blocked (Table 8). (ii) **H3 header manipulation: 99.2% blocked.** X-Forwarded-For injection, CF-Connecting-IP override attempts and Host header injection all failed—GUARDIAN trusts only CF-Connecting-IP from a loopback peer. (iii) **H4 behavioral mimicry: 66% blocked overall.** The 4-phase attack (legit handshake → enumeration → privilege escalation → injection) shows that GUARDIAN bans the source IP after the attack phase begins, retroactively blocking subsequent requests from that IP including its earlier "legit" requests in the same batch. (iv) **H5 timing evasion: 95% blocked.** Even slow probes staying under the authfail threshold (7 attempts vs. threshold of 8) were eventually caught by the global alarm when distributed across many sources. (v) **H6 resource exhaustion: 100% blocked.** Deep JSON nesting (up to 250 levels), 500-key objects, emoji bombs, multi-KB paths and connection churn were all stopped before reaching dispatch. (vi) **H8 IPv6 advanced: 99.9% blocked.** Multi-/64 rotation and expanded-notation encoding did not evade /64-keyed state. (vii) **H9 FP stress: the 90% HTTP block rate is a ban artifact.** All 400 H9 sources had already been used, and banned, by earlier hard-mode categories, and the benign sentences (e.g. "Can you select all the containers that dropped connections?") went to an exempt tool. Run directly through the detector, 6.7% of the 100,000 requests (3 of 45 distinct sentences) are false-blocked (Table 8). (viii) Latency remained flat at 2.6 ms p50 and 10.2 ms p99 across all categories, including oversized payloads.

10.3 Content-defense attribution

The stress harnesses record only the HTTP status. A BASTION block is a JSON-RPC error inside an HTTP 200 (Section 7), while an HTTP 403 is always a GUARDIAN decision. C10, H1 and H9 send every payload to shell_explain, which BASTION exempts from injection checks, and each suite runs its categories in sequence from one shared pool of documentation addresses: every C10 and H9 source had already appeared in an earlier, abusive category, while H1 ran first from fresh sources. The HTTP outcomes of these three categories therefore measure GUARDIAN ban carry-over, not content defense. To measure the content layer we run the exact payloads from the stress plans through BASTION's input check as a non-exempt tool (artifact/stress_bastion_direct.py). The check is deterministic, so each distinct payload is evaluated once and weighted by how often the plan sent it (Table 8).

By H1 sub-family the block rate is 100% for polyglots, 95.0% for SQL keyword obfuscation, 85.7% for traversal encodings, 85.0% for command-injection variants, 71.4%

Table 8: BASTION's input check run directly on the exact stress payloads (stress_bastion_direct.json).

Corpus	Sent	Blocked	Rate	Distinct
C10 injection	70,000	55,998	80.0%	21/27
C10 benign	30,000	0	0.0%	0/20
H1 WAF evasion	100,000	86,958	87.0%	60/69
H9 adversarial benign	100,000	6,667	6.7%	3/45

Distinct: distinct payloads blocked / total.

for Unicode SQL and 66.7% for double encoding. The misses are specific: tautology-only SQL (admin'–, ') OR ('1'='1), short shell chains without a listed dangerous command (| whoami, && ls -la /, python3 -c), zero-width and homoglyph characters inside OR, double URL encoding (%252e) and overlong UTF-8 (%c1%9c). The three H9 false positives name a system path (/etc/hosts, /proc/meminfo) or begin "Truncate the log file". Catching the misses needs input normalization (Unicode folding, repeated URL decoding) or broader patterns, and broader patterns would raise the false-positive rate that H9 measures.

Summary. Across 2,000,000 total requests, the admission gate (v1/v2) maintains **zero false blocks on authenticated benign MCP sessions** and blocks >99% of declared bots, credential probes, path scans, floods, IPv6 rotation and header manipulation. Measured directly on the stress corpora, BASTION's pattern-based content check blocks 80.0% of plain and 87.0% of WAF-evasion injection payloads, with 0% false positives on ordinary and 6.7% on adversarially worded benign inputs (Table 8). The residual misses and false positives are the expected boundary of regex detection over free-form tool arguments. A semantic-aware content filter (e.g. an LLM-based classifier) could narrow both, at a latency cost that would dominate the current sub-5 ms gate.

11 Statistical Rigor

Single-run latency numbers (E2) are reproducible but do not quantify measurement uncertainty. E6 repeats the latency measurement 5 times ($5 \times 500 = 2{,}500$ samples per condition) and reports 95% confidence intervals. The overhead of GUARDIAN on request latency is the difference between the "on" and "off" distributions, matched by rank.

Over 2,500 samples, median request latency is 0.121 ms with GUARDIAN disabled and 0.128 ms with GUARDIAN enabled (mean overhead 5.2 µs, 95% CI [5.0 µs, 5.4 µs]). The p95 latencies are 0.154 and 0.152 ms respectively, confirming that the tail is not widened. The overhead is operationally negligible: 5 µs is ≈0.001% of a typical tool execution time (seconds), and falls well within scheduling noise on any non-real-time system. We report the CI to bound the measurement, not to claim the difference matters.

12 Real-World Exposure

Method. We exposed the endpoint running v2 at its public hostname for a fixed window. The server logged, for every request, the timestamp, client address, user-agent, method, path, status, the outcome of the authentication check and the declared body length. These are enough to re-decide each request under any policy. The analysis tool

Table 9: Real traffic to the public endpoint (2026-09-30 to 2026-10-01, 23.0 h), operator traffic excluded. Classes are by user-agent.

Client class	Requests	Share	Sources
HTTP library / CLI	20,139	96.0%	4
Scanner (declared)	567	2.7%	2
Other bot	154	0.7%	2
Other	114	0.5%	4
MCP client	3	0.0%	1
Target			
/mcp	20,973	100.0%	
other paths (probes)	3	0.0%	
/	1	0.0%	

Table 10: The same real trace replayed through each policy on its own clock (replay reproduces the live server's status for 1211/1211 validation requests).

Policy	Blocked at gate	Reached auth	Non-client blocked
No gate (auth only)	0	20,972	0.0%
Guardian v1	16,870	4,102	80.4%
Guardian v2	20,950	22	99.9%

(artifact/analyze_trace.py) replays the trace in timestamp order on the trace's own clock through three policies: no gate (auth only), v1 and v2. Each runs its own unmodified code tree with isolated state. We validated the replay against ground truth: on 1211 requests from the v2 harness logs, it reproduced the live server's status code for every request (1211/1211, artifact/replay_fidelity.json). Operator traffic was excluded by user-agent. Only aggregates are published.

Results (RQ4). Over 23.0 h the endpoint received 20,977 requests from 5 sources (3 IPv6 /64s). The ten busiest sources sent 100% of the traffic (Table 9). Replayed through each policy (Table 10), v1 blocked 16,870 requests at the gate and v2 blocked 20,950. The share of non-MCP-client requests stopped before authentication was 80.4% and 99.9% respectively. With no gate, 20,972 requests reached the authentication layer, compared with 4,102 under v1 and 22 under v2. The composition and policy comparison are discussed in Tables 9 and 10.

Interpretation. The dominant traffic pattern was automated credential probing: a single source sent 20,000 requests to /mcp with bad tokens over the 23-hour window. Without the gate, all 20,972 of these reached the authentication layer and received a 401 for each guess. v1 banned the source after 8 failures but, because v1 keys state per IP, the 4,102 requests that arrived before banning were still answered—and the source was never banned again after the 60-minute ban expired. v2's global-alarm mechanism (keyed on /64 sources) tightened the threshold, banned the source on its 3rd failure in the alarm window, and blocked all subsequent requests: only 22 of 20,977 reached the authentication layer (22 against v1's 4,102). The difference between 80.4% and 99.9% non-client blocking is entirely attributable to v2's tighter per-source alarm and /64-keyed state. The 3 MCP-client requests (matching claude in the UA) were unaffected by either policy.

13 Related Work

MCP threat landscape. Hou et al. [12] give an early survey of MCP security threats and future directions. Huang et al. [13] apply STRIDE and DREAD threat modeling across five MCP components and identify tool poisoning as the most prevalent client-side vulnerability. The NSA has published a Cybersecurity Information Sheet on MCP security [22], warning that MCP's proliferation has outpaced the development of its security model and recommending controls including input validation, least privilege, sandboxed tool execution, rate limiting and audit logging. The Cloud Security Alliance provides complementary practitioner guidance [3]. OWASP catalogues broader LLM application risks [24], and Radosevich and Halloran [26] demonstrate major exploits against MCP-enabled LLM systems.

Post-authentication attacks. The largest body of MCP security research studies what an authorized or compromised client can do once inside. MCPTox [33] benchmarks tool poisoning across 45 live servers and 353 tools, finding attack success rates above 72%. MCP-ITP [17] automates implicit tool poisoning generation in a black-box setting. TrustShiftProbe [27] characterizes staged trust-escalation attacks. Indirect prompt injection [9] targets authorized data paths. Our threat model excludes these attacks on the model's reasoning: the admission layer (v1–v2) faces unauthenticated senders, and the post-authentication layers (v3–v5) bound what a token holder can send, receive and reach, not how the model interprets tool content.

Defense mechanisms. CASCADE [32] proposes a three-tiered local defense for prompt-injection detection in MCP systems (regex and entropy pre-filtering, semantic analysis, output filtering), evaluated on 5,000 samples at 95.85% precision.

Remote MCP deployments. Two recent measurement studies motivate this work. Zhou et al. [35] locate 7,973 reachable remote MCP servers and report that 40.55% of them answer tool calls from unauthenticated callers; for those that use OAuth, they classify nine kinds of authentication flaw. Sohail [29] characterizes 179 public remote endpoints and finds that whether a server authenticates is strongly correlated with its hosting platform rather than its operator's configuration. Both measure whether and how servers authenticate; we study what a single operator can place in front of the authentication layer, and measure it on a deployed endpoint.

Established network defense. Log-driven banning [7] and throttling of online guessing [8] are long established. Timing side-channels in string comparison are well understood [2]; constant-time comparison (`secrets.compare_digest` in Python, per the `hmac` module) is the standard mitigation. Crawler consent rests on convention [16], and restrictions aimed at AI crawlers have grown sharply [18]. Our S3 result confirms that convention-based identification cannot be relied on. Logrip [11] aggregates server-log activity across IP subnet classes to expose coordinated and distributed crawling that per-address tools miss; v2 applies the same insight online, keying admission state on network sources rather than individual addresses. Honeytokens—decoy credentials placed to detect unauthorized access—were introduced by Spitzner [30] and are available as a hosted service [31]. We apply the technique at the MCP output-filtering layer, replacing leaked secrets in tool responses with format-conforming canary credentials.

Comparison with alternatives. An operator can place several existing tools in front of an MCP endpoint: Cloudflare WAF rules or Bot Management (paid, closed-source, no MCP awareness), `nginx` rate limiting (per-IP, no auth-failure tracking, no ban persistence), or `fail2ban` scanning the access log (log-driven, works after the fact, no MCP-protocol awareness). Each addresses part of the threat surface, but none provides MCP-specific tool filtering, JSON-RPC-aware rate limiting, or an integrated evaluation against MCP client behaviour. Guardian is not a replacement for edge-layer defense; it is complementary. On this deployment, Cloudflare's free tier provides DDoS absorption and TLS termination; Guardian handles everything between the tunnel exit and the MCP dispatch.

Relationship to OAuth. The MCP specification defines an authorization flow built on OAuth 2.0 [10, 19]: a client discovers the server's authorization endpoint, obtains a scoped access token through a standard grant, and presents it as a bearer credential [15]. OAuth answers *"what is this authenticated client allowed to do?"*: it delegates identity verification to an authorization server, issues tokens scoped to specific capabilities, and assumes the client has been registered in advance. Guardian answers a different question—*"should this unknown sender's request reach the authentication layer at all?"*—using only behavioral signals available before any credential is checked: source address, request rate, user-agent signature, probe pattern and ban history. The two operate at different layers of the request lifecycle (Table 11). OAuth cannot stop a botnet from flooding the token endpoint or a scanner from probing paths that do not exist; Guardian cannot decide which tools a legitimate user should access. An MCP endpoint can use both: Guardian as the pre-authentication admission gate, and OAuth for identity, delegation and scope. In this deployment, the bearer token is a high-entropy static secret rather than an OAuth-issued token, because the endpoint has a single operator and no third-party clients; v3 scoped tokens (Section 6) approximate OAuth's per-resource scoping without requiring an authorization server. The evaluation results apply regardless of whether the bearer credential is a static secret or an OAuth access token: the gate runs before the credential is inspected, and Bastion's content defense runs after authentication succeeds, so the admission and content layers are credential-scheme agnostic.

Gap. Prior work on MCP security catalogues threats, studies post-authentication attacks, measures how public servers authenticate, or prescribes controls. We contribute a measured, before/after evaluation of a defense deployed in front of authentication on an Internet-facing MCP endpoint, show where per-address state fails, quantify the cost of fixes on identical workloads, and test robustness against adaptive and protocol-level attacks.

14 Discussion

Layered defense in practice. No single mechanism in this stack is individually novel. What the evaluation demon-

Table 11: Comparison of GUARDIAN and OAuth 2.0 on an MCP endpoint.

	GUARDIAN	OAuth 2.0
Layer	Admission (pre-auth)	Authorization (post-identity)
Question	Should this sender get a reply?	What can this client do?
Inputs	IP, UA, rate, probe pattern	Client ID, grant, scopes
Assumes	Nothing about the sender	Client is registered
Defends	Crawlers, scanners, probers, floods	Unauthorized actions by known clients
External deps	None (stdlib, in-process)	Authorization server, identity provider

strates is that their *combination*, applied specifically to an MCP endpoint and measured against MCP client behaviour, produces a defense with quantifiable properties: zero false blocks on benign sessions, measurable overhead (<10 µs), bounded resource consumption, and visible degradation when attackers adapt. Each v1 weakness traced to a single design choice (per-IP state keying) that produced measurable failures across multiple scenarios. The v2 fix was correspondingly local.

The batch bypass, closed. S19 revealed that JSON-RPC batch requests let an authenticated client bypass the per-request rate limit entirely. v4 BASTION closes this with a batch-size cap (default 20 messages per POST): batches exceeding the cap are rejected before dispatch. This trades strict JSON-RPC batch compliance for safety. In our deployment logs, all observed MCP client implementations (Claude Code, MCP Inspector, `python-httpx`) sent one message per POST; the cap of 20 is conservative.

Input validation vs. natural language. The central tension in Layer 4 is that some tools accept free-form text (e.g. `devops_ask` takes natural-language questions about servers). Naive keyword matching would false-block "why is the database dropping connections?" as SQL injection. The solution has two parts: structural patterns that require injection *syntax* (e.g. `DROP TABLE`, not just `drop`), and per-tool exemptions for tools whose purpose is to receive shell syntax. S23, E9 and the C10 corpus confirm zero false positives on ordinary natural-language inputs (3, 50 and 30,000 requests), adversarially worded inputs reach 6.7% (H9, Table 8), and S24 confirms that exempt tools pass injection payloads by design.

Active deception as a defense layer. Honeytokens shift output filtering from passive (redact and forget) to active (replace and track). The key design constraint is that a legitimate client should never see a honeytoken under normal operation: tool responses should not contain secrets, so the honeytoken fires only on the same responses that would have been redacted. The canary callback gives the operator something passive redaction cannot: attribution of the attacker's infrastructure when an exfiltrated credential is reused. The limitation is that credential-shaped honeytokens only produce callbacks when the attacker actually tries the credential against a service that reports the attempt (e.g. AWS CloudTrail for canary AWS keys) or visits the callback URL [31]. An attacker who collects but never uses the token is not detected.

Generalizability. This study covers one endpoint, one operator, one tool catalog. The architecture is not specific to this deployment—it requires only HTTP headers, a bearer token and a tool registry with JSON Schema—but the scenario set, default thresholds and performance numbers are specific to the platform. Other MCP deployments may face different traffic patterns, use different transport modes (SSE, stdio proxy), or sit behind different CDNs with different header semantics.

15 Limitations

This is a single-system, single-operator study. Controlled experiments ran on loopback with an emulated Cloudflare hop, so edge behaviour is assumed rather than measured, and the latency figures exclude network and tunnel time. The benign workload covers concurrent multi-client sessions and rate-window recovery (S13, S14) but not SSE streaming or mixed read/write tool workloads. The v2 defaults (the /64 prefix, the 20k cap and the global-alarm thresholds), v3 defaults (30 calls per tool per window) and v4 defaults (injection patterns, batch cap, enumeration thresholds) are reasonable choices, not tuned optima.

BASTION's input validation is pattern-based: it catches structural injection syntax but not every obfuscation. Measured directly on the stress corpora (Table 8), it misses 13.0% of WAF-evasion payloads (zero-width and homoglyph substitution, double URL encoding, overlong UTF-8, interpreter-wrapped shell chains) and 20.0% of plain injection payloads (tautology-only SQL, short shell chains), and false-blocks 6.7% of adversarially worded benign requests. The stress suites sent these payloads to an injection-exempt tool and recorded only HTTP status, so their C10, H1 and H9 outcomes measure GUARDIAN, not BASTION (Section 10.3); the direct measurement replaces them but does not exercise the HTTP path end to end. Output filtering catches nine known secret formats but cannot detect proprietary or novel credential patterns. Behavioral analysis uses fixed thresholds; adaptive adversaries who stay just under the enumeration or cycling limit are not detected. Honeytoken injection produces attribution only when the attacker reuses the credential against a service that reports the attempt; silent collection is not detected. Honeytokens for key-value patterns (passwords, API keys) preserve the surrounding structure but change the value format, which could alert a sophisticated attacker that the credential is fake.

The real-traffic window covers a single hostname whose discoverability depends on certificate-transparency logs and DNS enumeration by third parties. The techniques evaluated (rate limiting, UA signature blocking, auth-failure banning, token scoping, injection detection, output redaction) are individually well established; the contribution is their measured application and failure modes in the MCP context, not the techniques themselves.

16 Conclusion

A few hundred lines of dependency-free code in front of an MCP endpoint, together with a strong token and a minimized tool surface, measurably stop declared crawlers, cre-

dential probing, path scanning and floods at microsecond cost. The first version failed where its state was keyed on individual addresses and allowed to grow without bound. Distributed and rotating adversaries slipped through, and a growing ban list turned the defense into a latency liability. Keying on network sources and bounding state fixed each measured failure without new false blocks. Extending defense past authentication with scoped tokens and per-tool rate limits bounds the blast radius of a leaked credential and prevents authorized abuse of individual tools. Bastion closes the remaining post-auth surface: injection patterns and schema validation catch malicious tool arguments before execution, behavioral analysis detects reconnaissance and enumeration, output filtering redacts secrets from tool responses, and an optional honeytoken mode replaces redacted secrets with trackable canary credentials that attribute credential exfiltration when the attacker reuses them. Together, Guardian and Bastion span the full request lifecycle from first byte to final response in 32 targeted scenarios, six advanced robustness scenarios, and 2,000,000 stress-test requests across 20 traffic categories. The stress evaluation confirms zero false blocks on benign sessions across all volumes. Run directly on the same corpora, Bastion's pattern-based check stops 87% of standard WAF-evasion payloads at a 6.7% false-positive rate on adversarially worded benign inputs; the residue (Unicode homoglyphs, double encoding, tautology-only SQL) marks where a semantic-aware filter is needed. Mirage replaces the binary post-authentication decision with graded, verifiable routing and, on a deployed service, contained a low-and-slow adversary after 8 calls with no false action on 1,800 benign requests. Self-hosted agent endpoints should treat such gates as cost and visibility controls, and never as the authentication boundary.

Artifact. artifact/: harness2.py, harness.py and surface.py (tool surface), analyze_trace.py, gen_realtraffic_tex.py, results_v1.json, results_v2.json, results_v3.json, results_v4.json, surface.json, replay_fidelity.json, and aggregate real_traffic.json; fp_corpus.py (50-input false-positive corpus and 30-payload evasion suite), fp_results.json; megaharness.py (1,000,000-case standard stress suite, 10 categories), megaharness_hard.py (1,000,000-case hard-mode evasion suite, 10 categories), megaharness_summary.json, megaharness_hard_summary.json; stress_bastion_direct.py and stress_bastion_direct.json (content check on the exact stress payloads); gen_mirage_tex.py and aggregate Mirage evidence (public-data/mirage_evidence_report.json); agentkit/bastion.py (post-auth defense module); https://github.com/TGvenomYT/agent-endpoint-defense.

Ethics. Controlled experiments targeted only the author's own software, using documentation-range addresses. The public exposure passively logged requests sent to the author's own hostname. No third-party system was probed and no traffic source was attacked. Raw logs, which contain client IP addresses, are kept private and deleted after analysis. Only aggregates are released.

Use of generative AI. This work made substantial use of a large language model, Anthropic's Claude, through the Claude Code tool. It helped design and implement the defense code, wrote much of the evaluation and stress harnesses and analysis scripts, drafted and edited much of the manuscript, and checked the references. The experiments ran on the author's own systems, and every reported number is read from the released code and result files rather than produced by the model. The author directed the work, reviewed the manuscript, and takes full responsibility for its content.

References

[1] J. Arkko, M. Cotton, and L. Vegoda. IPv4 address blocks reserved for documentation. RFC 5737, IETF, 2010.

[2] David Brumley and Dan Boneh. Remote timing attacks are practical. In *12th USENIX Security Symposium*, 2003.

[3] Cloud Security Alliance. Agentic MCP security best practices guide. https://labs.cloudsecurityalliance.org/agentic/agentic-mcp-security-best-practices-v1/, 2026.

[4] Cloudflare. HTTP request headers: CF-Connecting-IP. https://developers.cloudflare.com/fundamentals/reference/http-headers/, 2025.

[5] Cloudflare. Cloudflare tunnel documentation. https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, 2025.

[6] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. ZMap: Fast internet-wide scanning and its security applications. In *22nd USENIX Security Symposium*, 2013.

[7] Fail2ban contributors. Fail2ban: Daemon to ban hosts that cause multiple authentication errors. https://github.com/fail2ban/fail2ban, 2024.

[8] Paul A. Grassi, James L. Fenton, Elaine M. Newton, et al. Digital identity guidelines: Authentication and lifecycle management. Technical Report SP 800-63B, National Institute of Standards and Technology, 2017.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[10] D. Hardt. The OAuth 2.0 authorization framework. RFC 6749, IETF, 2012.

[11] Rama Carl Hoetzlein. Protecting small organizations from AI bots with Logrip: Hierarchical IP hashing. *arXiv preprint arXiv:2508.03130*, 2025.

[12] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, 35(10):1–37, 2026. doi: 10.1145/3796519.

[13] Charoes Huang, Xin Huang, Ngoc Phu Tran, and Amin Milani Fard. Model context protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning. *arXiv preprint arXiv:2603.22489*, 2026.

[14] G. Huston, A. Lord, and P. Smith. IPv6 address prefix reserved for documentation. RFC 3849, IETF, 2004.

[15] M. Jones and D. Hardt. The OAuth 2.0 authorization framework: Bearer token usage. RFC 6750, IETF, 2012.

[16] M. Koster, G. Illyes, H. Zeller, and L. Sassman. Robots exclusion protocol. RFC 9309, IETF, 2022.

[17] Ruiqi Li, Zhiqiang Wang, Yunhao Yao, and Xiang-Yang Li. MCP-ITP: An automated framework for implicit tool poisoning in MCP. *arXiv preprint arXiv:2601.07395*, 2026.

[18] Shayne Longpre, Robert Mahari, Ariel Lee, et al. Consent in crisis: The rapid decline of the AI data commons. *arXiv preprint arXiv:2407.14933*, 2024.

[19] Model Context Protocol. MCP authorization specification (2025-06-18). https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization, 2025.

[20] Model Context Protocol. Model context protocol specification (2025-06-18). https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[21] T. Narten, G. Huston, and L. Roberts. IPv6 address assignment to end sites. RFC 6177, IETF, 2011.

[22] National Security Agency. Model context protocol (MCP): Security design considerations for AI-driven automation. Cybersecurity Information Sheet U/OO/6030316-26, National Security Agency, May 2026.

[23] M. Nottingham and R. Fielding. Additional HTTP status codes. RFC 6585, IETF, 2012.

[24] OWASP Foundation. OWASP top 10 for large language model applications (2025). https://genai.owasp.org/llm-top-10/, 2024.

[25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[26] Brandon Radosevich and John Halloran. MCP safety audit: LLMs with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*, 2025.

[27] Mehrdad Rostamzadeh, Sidhant Narula, Mohammad Ghasemigol, and Daniel Takabi. TrustShiftProbe: Characterizing, benchmarking, and defending staged trust attacks on MCP servers. *arXiv preprint arXiv:2608.23763*, 2026.

[28] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[29] Muhammad Abdullah Sohail. Characterizing network centralization and observability in the remote MCP ecosystem. *arXiv preprint arXiv:2609.19100*, 2026. Accepted at the 1st IEEE ICNP Workshop on Network Infrastructure and Protocols for AI Agents (NIPA 2026).

[30] Lance Spitzner. Honeytokens: The other honeypot. https://community.broadcom.com/symantecenterprise/communities/community-home/librarydocuments/viewdocument?DocumentKey=5c9c1f09-4115-4c02-8bf7-e6290cfb8a47, 2003. SecurityFocus.

[31] Thinkst Applied Research. Canarytokens: A free, quick, painless way to help defenders discover they've been breached. https://canarytokens.org, 2024.

[32] İpek Abasıkeleş Turgut and Edip Gümüş. CASCADE: A cascaded hybrid defense architecture for prompt injection detection in MCP-based systems. In *The 12th International Conference on Computing and Artificial Intelligence (ICCAI)*, 2026. arXiv:2604.17125 (v1).

[33] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A benchmark for tool poisoning attack on real-world MCP servers. *arXiv preprint arXiv:2508.14925*, 2025.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[35] Huijun Zhou, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang. A first measurement study on authentication security in real-world remote MCP servers. *arXiv preprint arXiv:2605.22333*, 2026.